บทพูดนำเสนอ Colorado Back-Office System

01418321 System Analysis and Design หมู่ 200 · นำเสนอ 40 นาที
ระบบจัดการหลังบ้านอัตโนมัติ เฟส 1: Rakuten Ichiba, Rakuten RMS และ RSL

แบ่งกันพูดอย่างไร

ผู้พูด	สไลด์	เรื่องที่พูด	เวลา
นายธนภัทร พูลโภคา	1–14	เปิดเรื่องและแนะนำธุรกิจเป็นภาษาอังกฤษ ปัญหา วัตถุประสงค์ ขอบเขต Business Process เดิมและใหม่ Use Case Diagram และ CRUD	10:15
นายอชิระ ภู่รุ่งเรือง	15–33	วิธีอ่าน Sequence แบบ MVC แล้ว Use Case ทั้ง 17 ตัว Description Sequence Collaboration (3S, 5A, 6A, 7S แบบละเอียด)	11:35
นางสาวปิยากร ผลพานิช	34–57	ER ตาราง Class State 5 คลาส สถาปัตยกรรม เว็บจริง ความปลอดภัย หน้าจอ รายงาน SQL การทดสอบ สาธิต + SELECT สรุป	14:35
	รวม	เฉพาะหัวข้อ **พูด** รวมสาธิต + SELECT 4:00 (ไม่รวมพูดเพิ่ม)	**36:25**
		ถาม-ตอบ (ดู `qna_prep/qna-Colorado.pdf`)	ประมาณ 4:00

วิธีใช้บทนี้

- แต่ละคนเริ่มหน้าใหม่และมีสีของตัวเอง หัวกล่องบอกเลขสไลด์ ชื่อบนสไลด์ และเวลาโดยประมาณ ในไฟล์สไลด์มีบทพูดย่อในช่อง notes ทุกหน้า
- **พูด** เป็นประโยคที่พูดตามได้ทันที **พูดเพิ่มถ้ามีเวลา** ใช้เมื่อเวลาเหลือหรืออาจารย์สนใจ ถ้าเวลาไม่พอให้ตัดส่วนนี้ก่อน
- **วิธีพูด** เป็นคำแนะนำ ไม่ต้องพูดออกมา **ชี้จอ** บอกว่าให้ชี้หรือวนเมาส์ตรงไหน
- ตัวเลขที่เป็น**ตัวหนาสี**ต้องพูดให้ตรง ตัวเลขเวลาของงานมือเป็น**ค่าประมาณ**จากที่เจ้าของเล่า ต้องพูดคำว่า “ประมาณ” ทุกครั้ง
- ตามเกณฑ์ของวิชา ต้องแนะนำธุรกิจเป็นภาษาอังกฤษ บทนี้จึงให้พูดภาษาอังกฤษในสไลด์ 1 ถึง 3 แล้วเปลี่ยนเป็นภาษาไทยตั้งแต่สไลด์ 4
- จบส่วนของตัวเองแล้วพูดประโยคใน**ส่งต่อ** คนถัดไปรับด้วย “ขอบคุณครับ/ค่ะ” แล้วพูดต่อทันที

ประโยคที่ทุกคนต้องพูดได้

ระบบรับงานซ้ำไป 7 Use Case แต่ทุกขั้นที่ใช้เงินจริง คือการตัดสินใจใน 5A และการสั่งซื้อกับรับของใน 6A ยังต้องให้เจ้าของกดเองเสมอ

เวลาต่อสไลด์

สไลด์	ชื่อบนสไลด์	ผู้พูด	เวลา	เวลาสะสม
1	Cover (EN)	คนที่ 1	0:30	0:30
2	Agenda (EN)	คนที่ 1	0:30	1:00
3	Business introduction (EN)	คนที่ 1	1:10	2:10
4	Problems	คนที่ 1	1:00	3:10
5	Objectives	คนที่ 1	0:45	3:55
6	Scope	คนที่ 1	0:45	4:40
7	Business process: As-Is	คนที่ 1	0:45	5:25
8	Pain points	คนที่ 1	0:45	6:10
9	Business process: To-Be	คนที่ 1	0:55	7:05
10	To-Be: who does what	คนที่ 1	0:35	7:40
11	Business process to use case	คนที่ 1	0:35	8:15
12	Use case diagram: before	คนที่ 1	0:45	9:00
13	Use case diagram: reviewed	คนที่ 1	0:35	9:35
14	CRUD table	คนที่ 1	0:40	10:15
15	17 use cases	คนที่ 2	0:40	10:55
16	One order, end to end	คนที่ 2	0:45	11:40
17	Sequence diagram: MVC	คนที่ 2	1:00	12:40
18	1A Log in	คนที่ 2	0:40	13:20
19	2A · 2S · 3A	คนที่ 2	0:40	14:00
20	3S Check stock	คนที่ 2	0:55	14:55
21	3S Sequence diagram	คนที่ 2	0:35	15:30
22	3S Collaboration diagram	คนที่ 2	0:20	15:50
23	1S · 6S · 9A Labels	คนที่ 2	0:40	16:30
24	4S Unit cost	คนที่ 2	0:35	17:05
25	5A Decide reorder	คนที่ 2	0:40	17:45
26	5A Sequence diagram	คนที่ 2	0:30	18:15
27	5A Collaboration diagram	คนที่ 2	0:20	18:35
28	6A Reorder and receive	คนที่ 2	0:45	19:20
29	6A Sequence diagram	คนที่ 2	0:30	19:50
30	6A Collaboration diagram	คนที่ 2	0:15	20:05
31	8A · 5S · 7S Ship and deduct	คนที่ 2	0:45	20:50
32	7S Sequence diagram	คนที่ 2	0:25	21:15
33	4A · 7A · 10A	คนที่ 2	0:35	21:50
34	ER diagram	คนที่ 3	0:40	22:30
35	ER diagram with queries	คนที่ 3	0:20	22:50
36	Tables and queries	คนที่ 3	0:30	23:20
37	Class diagram	คนที่ 3	0:30	23:50
38	State diagram: Order (1/3)	คนที่ 3	0:35	24:25
39	State diagram: Order (2/3)	คนที่ 3	0:30	24:55
40	State diagram: Order (3/3)	คนที่ 3	0:20	25:15
41	State diagram: Reorder	คนที่ 3	0:30	25:45
42	State diagram: Label · SKU	คนที่ 3	0:30	26:15
43	State diagram: delivery · sync	คนที่ 3	0:30	26:45
44	Layers of the web app	คนที่ 3	0:40	27:25
45	executeWorkflow	คนที่ 3	0:45	28:10
46	Locks and double deduction	คนที่ 3	0:40	28:50
47	External systems	คนที่ 3	0:15	29:05
48	Security	คนที่ 3	0:25	29:30
49	Platform and components	คนที่ 3	0:10	29:40
50	Site map and screens	คนที่ 3	0:25	30:05
51	Output design	คนที่ 3	0:20	30:25
52	SQL results	คนที่ 3	0:20	30:45
53	Testing	คนที่ 3	0:35	31:20
54	Live demo + SELECT	คนที่ 3	4:00	35:20
55	Team and responsibilities	คนที่ 3	0:20	35:40
56	Summary	คนที่ 3	0:35	36:15
57	Q&A	คนที่ 3	0:10	36:25

ถ้าเวลาไม่พอ ตัดตามลำดับนี้

ลำดับ	ตัดอะไร	ได้เวลาคืน
1	ตัดหัวข้อ**พูดเพิ่มถ้ามีเวลา**ทุกสไลด์	2–3 นาที
2	สาธิต (สไลด์ 54) เหลือข้อ 1–3: Login, นำเข้า, ยืนยัน ORD-24194 พร้อม SELECT ทุกข้อ (ห้ามข้าม SELECT)	1:30
3	สไลด์ Collaboration 22, 27, 30 และ State 42–43 พูดประโยคเดียว (Sequence ห้ามตัด)	1:00
4	สไลด์ 10 (16 ขั้นตอน) และ 11 (BP กับ UC) พูดแค่ประโยคแรก	0:45
5	สไลด์ 49 (Platform), 51 (รายงาน), 47 (ระบบภายนอก) พูดประโยคเดียว	0:45
6	สไลด์ 48 (Security) พูดแค่ scrypt กับ hash ของ session	0:20

ห้ามตัด (อาจารย์เน้น Sequence, ER, Query, State, เดโม + SELECT): สไลด์ 3 (ภาษาอังกฤษ), 17 (Sequence แบบ MVC), 21, 26, 29, 32 (Sequence), 34–36 (ER และ Query), 38–41 (State), 54 (เดโม + SELECT)

ข้อเท็จจริงที่ต้องพูดให้ตรง

- **17 Use Case** (Admin เริ่ม 10 ตัว ระบบทำ 7 ตัว) **17 ตาราง** ใน PostgreSQL จาก migrations 001–008, **22 relationship**, SQL **113 คำสั่ง** รันผ่านครบ ทดสอบบนเว็บจริง **36 กรณี** ผ่านครบ
- เว็บอ่านเขียน **PostgreSQL จริงทุกหน้า** ผ่าน Server Action → use-case → queries → db.ts **ไม่ใช่ localStorage** และไม่มีคลาส Controller, Repository หรือ Scheduler ใน Sequence ชื่อ Screen = page (ชั้น View ของ MVC), Controller = Server Action (เป็นบทบาท ไม่ใช่คลาสใหม่) และ**ไม่มีลูกศรกลับหา Actor**
- Rakuten RMS, RSL, ขนส่ง, เครื่องพิมพ์, Label Service และ Supplier ต่อผ่าน adapter ซึ่ง**ตอนนี้เป็นตัวจำลอง (mock)** กรณีระบบภายนอกล่มจำลองด้วยหน้า `/dev`
- รายงานรอบแรกออกแบบ 15 ตาราง ฉบับสุดท้ายเป็น 17 ตามโค้ดจริง ถ้าถูกถามให้ตอบตาม qna
- ยังไม่มี unit test หรือ e2e test อัตโนมัติ ในโค้ดมี lint, type check, build, smoke test ของการ reset ส่วน SQL ทุกข้อรันด้วย `npm run sql` ในโฟลเดอร์เครื่องมือของเล่ม `04-chapter2-diagrams-database` (ไม่ได้อยู่ใน repo เว็บ)
- ผลฝั่ง “After” ในสไลด์ 56 เป็น**เป้าหมาย** ยังไม่ได้วัดกับร้านจริง เงินทุกจำนวนเป็นเยน

เตรียมก่อนวันนำเสนอ

- คนที่ 3 รัน `docker start rsl-pg`, `npm run db:reset`, `npm run dev` ก่อนเริ่ม 10 นาที แล้วเข้าสู่ระบบที่ `http://localhost:3000` ด้วย `admin@colorado.jp` (รหัสผ่านจาก `SEED_ADMIN_PASSWORD` ใน `.env.local`)
- ห้ามกดอะไรบนเว็บหลัง reset จนถึงตอนสาธิต เพราะ ORD-24194 ต้องยังอยู่ที่ “รอตรวจสอบคำสั่งซื้อ” (อย่าเปิดหน้า `/shipping` ก่อนเดโม เพราะจะดึงสถานะขนส่งและรัน 6S)
- เปิด terminal ที่สองค้างไว้: `docker exec -it rsl-pg psql -U rsl_app -d rsl_fulfillment_hub`

(ถ้าใช้ PostgreSQL ที่ติดตั้งเอง ใช้ `psql -U rsl_app -d rsl_fulfillment_hub` แทน) พิมพ์ SELECT ของสไลด์ 54 ไว้ในไฟล์ text เพื่อ copy วาง ซ้อม SELECT ก่อนพรี 1 รอบ

- ขยายตัวอักษร terminal ให้ใหญ่ อาจารย์ต้องอ่านผล SELECT ได้ ใช้ `\x auto` ถ้าแถวยาว

นายธนภัทร พูลโภคา

สไลด์ 1 ถึง 14

เวลาประมาณ 10 นาที 15 วินาที

สไลด์ 1 Cover (English)

ประมาณ 30 วินาที

พูด

Good morning, Ajarn Somchoke, and everyone. Our project is the Colorado Back-Office System, a back-office system for Colorado Company Limited. My name is นายธนภัทร พูลโภคา, and my teammates are นายอชิระ ภู่รุ่งเรือง and นางสาวปิยากร ผลพานิช.

Today we present phase one. It works with Rakuten Ichiba, Rakuten RMS and Rakuten Super Logistics, and it automates three things: shipping labels, reordering and unit cost.

ชี้จอ วนเมาส์ผ่านป้าย 3 อัน Auto label, Auto reorder, Unit cost calculator ภาพขวาคือ Dashboard ของเว็บจริง

สไลด์ 2 Agenda (English)

ประมาณ 30 วินาที

พูด

The talk has three parts. I will introduce the business, the problems, the business process, the use case diagram and the CRUD table. Achira will walk through all seventeen use cases with their sequence and collaboration diagrams. Piyakorn will cover the database, the architecture of the real web application, testing and a short live demo.

ชี้จอ กล่อง 01 02 03 ทีละกล่อง แล้วแถบล่าง 17 / 17 / 113 / 36

สไลด์ 3 Business introduction (English)

ประมาณ 1 นาที 10 วินาที

พูด

Colorado Company Limited sells imported and OEM products on Rakuten Ichiba in Japan: fishing gear and lures, outdoor goods, imported household items and Thai herbal inhalers.

The business started in 2009 as a side business. Today one owner still runs almost everything. Rakuten Super Logistics, or RSL, keeps most of the stock and ships the parcels, and the company also has a small warehouse of its own.

On the right is one order today. The red steps are done by hand: checking stock in two warehouses, matching the order with RSL, formatting and printing the label, and sending the tracking number. That is four of six steps.

Our system takes over these repetitive steps, but the owner still approves every decision that spends money. Phase one covers only Rakuten. Yahoo! Auctions and Amazon come later.

ตัวเลขที่ต้องเน้น **2009**, **one owner**, **4 of 6 steps** by hand

วิธีพูด หยุดสั้นๆ ก่อนประโยค “the owner still approves every decision that spends money”

เชื่อมไปสไลด์ถัดไป ``From here, we will continue in Thai.'' “มาดูว่างานมือเหล่านี้ใช้เวลาเท่าไหร่ครับ”

สไลด์ 4 Problems — ประมาณ 1 นาที

พูด

- ร้านขายประมาณ **5 ถึง 10 ออเดอร์ต่อวัน** ทุกออเดอร์ผ่านงานมือ 5 อย่างตามกราฟ ตรวจคำสั่งซื้อ 2 นาที เช็กสต๊อก 2 คลัง 3 นาที จับคู่ RSL 2 นาที จัดรูปแบบและพิมพ์ใบปะ 3 นาที แจ้งเลขติดตาม 2 นาที
- รวมประมาณ **12 นาทีต่อออเดอร์** หรือ 60 ถึง 120 นาทีต่อวัน เป็นค่าประมาณจากที่เจ้าของเล่า ยังไม่ได้จับเวลาจริง
- ยังเสียเงินด้วย ส่งเองประมาณ 600 เยนต่อชิ้น คู่แข่งประมาณ 200 เยน ต่างกัน **400 เยน** และต้นทุนจริง 1 SKU ต้องรวม **8 รายการ** เอง ใช้ประมาณ 20 ถึง 30 นาที การสั่งซื้อเติมจึงใช้ความรู้สึก

พูดเพิ่มถ้ามีเวลา

ปัญหาไม่ใช่ “ไม่มีคอมพิวเตอร์” เจ้าของใช้ Rakuten RMS อยู่แล้ว ปัญหาคืองานซ้ำที่ไม่มีเกณฑ์และวัดผลได้

สไลด์ 5 Objectives — ประมาณ 45 วินาที

พูด

- วัตถุประสงค์มี 3 เรื่อง Auto label ใบปะพร้อมทันทีเมื่อรู้แหล่งส่ง Auto reorder เช็กสต๊อก 2 คลังและเตรียมใบสั่งซื้อให้เจ้าของอนุมัติ และ Unit cost คำนวณต้นทุนด้วยสูตรเดียวทุกครั้ง แต่ละเรื่องผูกกับ Use Case ที่เขียนกำกับไว้
- ตัวชี้วัดใน 3 เดือน ทุก Order ได้ใบปะถูกต้องโดยไม่จัดรูปแบบเอง ทุก SKU ที่ต่ำกว่าเกณฑ์ถูกแจ้ง คำนวณต้นทุนไม่เกิน **1 นาที** และเวลางานหลังบ้านลดลงอย่างน้อย**ครึ่งหนึ่ง**

สไลด์ 6 Scope — ประมาณ 45 วินาที

พูด

- เฟส 1 คือ Rakuten Ichiba, RMS และ RSL เท่านั้น คลังมี 2 แห่ง คือ RSL กับคลังบริษัท ระบบภายนอกต่อผ่าน adapter ซึ่งตอนนี้เป็นตัวจำลอง
- นอกขอบเขต คือ Amazon, Yahoo, การรับเงิน การคุยกับลูกค้า และ 2FA
- หลักการสำคัญคือ human-in-the-loop ระบบทำงานซ้ำ แต่ 5A ตัดสินใจ และ 6A สั่งซื้อกับรับของ เจ้าของต้องกดเอง 7A กับ 10A ต้องใส่เหตุผลหรือเก็บ snapshot ก่อนเสมอ

สไลด์ 7 Business process: As-Is — ประมาณ 45 วินาที

พูด

- Business Process เดิมมี 15 ขั้น อ่านจากบนลงล่าง ซ้ายไปขวา Super Admin ทำเองทุกขั้น รับ Order ตรวจ เช็กสต๊อก จับคู่ RSL พิมพ์ Label ส่งมอบ แจ้งเลขติดตาม
- ถ้าสต๊อกไม่พอ คำนวณต้นทุนเอง คุ้มค่าสั่ง Supplier ไม่คุ้มหรือ Supplier ไม่มีของ ยกเลิก Order เมื่อของมาก็วนกลับไปจับคู่ RSL

ชี้จอ ไล่นิ้วตามเส้นหลักในรูป แล้วชี้ทางแยกสต๊อกไม่พอ

สไลด์ 8 Pain points — ประมาณ 45 วินาที

พูด

- จุดสีแดงคือปัญหา 4 จุด วัดได้เป็นเวลาทั้งหมด จับคู่สต๊อกประมาณ 5 นาทีต่อออเดอร์ คำนวณความคุ้มค่า 20 ถึง 30 นาทีต่อครั้ง จัดรูปแบบใบปะ 3 นาที ติดตามพัสดุ 2 นาที
- ทุกจุดวัดซ้ำได้หลังใช้ระบบ จึงใช้ตรวจว่าปัญหาลดลงจริง

สไลด์ 9 Business process: To-Be — ประมาณ 55 วินาที

พูด

- BP ใหม่เพิ่ม lane ระบบ ระบบรับงานซ้ำ **7 Use Case** 1S ถึง 7S เช่น จับคู่กฎ SKU เลือกแหล่งสต๊อก จัดรูปแบบใบปะ แจ้งเลขติดตาม ตัดสต๊อก
- คนยังทำงานที่ต้องตัดสินใจหรือเป็นงานกายภาพ เช่น ตรวจ Order ตั้งกฎ SKU อนุมัติสั่งซื้อ รับของ พิมพ์ใบปะ ส่งมอบพัสดุ
- จุดที่เปลี่ยนจากรอบก่อนคือทางแยกหลัง 3S ถ้า RSL พอไปจับคู่ RSL ถ้า RSL ไม่พอแต่คลังบริษัทพอ บริษัทส่งเองและไปจัดรูปแบบใบปะทันที ถ้าไม่มีแหล่งใดพอไปคำนวณต้นทุนและให้เจ้าของตัดสินใจ

ชี้จอ ทางแยก “RSL พอ” “คลังบริษัทพอ” “ไม่มีแหล่งใดพอ” ในรูป

สไลด์ 10 To-Be: who does what — ประมาณ 35 วินาที

พูด

- ตารางนี้คือ 2.5 ครบ 16 ขั้น คอลัมน์ผู้ทำ A คือระบบ M คือคน คอลัมน์ขวาบอกว่าขั้นนั้นแก้ปัญหาข้อไหนใน 2.3
- ขั้น 13 คือ 5A เป็น manual โดยตั้งใจ (M*) เพราะผูกกับเงิน ขั้น 16 คือ 10A ลบข้อมูลเก่า ที่เพิ่มตามคำแนะนำของอาจารย์

สไลด์ 11 Business process to use case — ประมาณ 35 วินาที

พูด

- รูปนี้คือ BP To-Be ที่กำกับเลข Use Case ไว้บนทุกกล่อง ทั้งงานคนและงานระบบ ตารางขวาจับคู่ทีละแถว ทุก Use Case ย้อนกลับไปหาขั้นใน BP ได้
- ยกเว้น 1A ซึ่งเป็น precondition และ 10A ซึ่งเป็นงานดูแลระบบ

สไลด์ 12 Use case diagram: before — ประมาณ 45 วินาที

พูด

- Use Case Diagram แรกได้จาก BP เดิมตรงๆ มี **9 Use Case** เกือบทั้งหมดผูกกับ Super Admin
- พอทำ CRUD ก่อนปรับปรุง เห็นช่องว่าง 5 เรื่อง ไม่มีกฎ SKU ไม่มีการตรวจ Order การจัดรูปแบบกับพิมพ์ใบปะปนกัน งานหลังส่งมอบไม่มีใครรับ และไม่มีใครลบข้อมูลเก่า แต่ละเรื่องเพิ่ม Use Case ตามที่เห็นทางขวา

สไลด์ 13 Use case diagram: reviewed — ประมาณ 35 วินาที

พูด

- หลังทบทวนมี **17 Use Case** เพิ่มใหม่ **7 ตัว** คือ 2A, 3A, 9A, 10A, 2S, 5S และ 7S
- A คือ Super Admin เริ่ม 10 ตัว S คือระบบทำ 7 ตัว actor ภายนอกมี Marketplace, Supplier, Delivery และ RSL ส่วน 1S, 4S, 5S มีปุ่มให้กด แต่ขั้นทั้งหมดระบบทำ จึงไม่ลากเส้นจาก Super Admin

สไลด์ 14 CRUD table — ประมาณ 40 วินาที

พูด

- CRUD ที่ทบทวนแล้วสร้างจาก SQL ทุกข้อใน Use Case Description INSERT SELECT UPDATE DELETE กลายเป็น C R U D จึงมี **17 entity** ตรงกับตารางจริงและ ER ทุกตัว
- แถวสีแดงคือ Use Case ที่เพิ่ม ตัว D สีแดงเกือบทั้งหมดอยู่ที่ 10A ซึ่งเป็น Use Case หลักที่ลบข้อมูล

ส่งต่อ “ต่อไปขอส่งให้นายอชิระ ภู่รุ่งเรือง เล่ารายละเอียดของ Use Case ทั้ง 17 ตัวครับ”

นายอชิระ ภู่รุ่งเรือง

สไลด์ 15 ถึง 33

เวลาประมาณ 11 นาที 35 วินาที

สไลด์ 15 17 use cases

ประมาณ 40 วินาที

พูด

- ซ้ายคือ 10 Use Case ที่ Admin เริ่ม ขวาคือ 7 ตัวที่ระบบทำ คอลัมน์ “เริ่มเมื่อ” บอกว่าในเว็บจริงทำงานตอนไหน เช่น 2S ทำต่อจาก 2A ทันที 7S ทำต่อจาก 5S
- คอลัมน์ Q คือจำนวน query รวม **113 query** เลข `Q<UC>.n` เรียงตามขั้นตอน
- ในเล่มทุก Use Case มี Description แล้วตามด้วย Sequence และ Collaboration ทันที วันนี้จะลงละเอียด 3S, 5A, 6A และ 7S ที่เหลือพูดเป็นกลุ่ม

สไลด์ 16 One order, end to end

ประมาณ 45 วินาที

พูด

- ภาพรวม Order หนึ่งรายการ 4A นำเข้า 2A ตรวจ แล้วระบบทำ 2S 3S 1S 6S ต่อทันที กล่องสีฟ้าคือระบบทำ Admin พิมพ์ 9A ส่งมอบ 8A ระบบแจ้งเลขติดตาม 5S และตัดสต๊อก 7S ใต้กล่องคือสถานะหลังขั้นนั้น
- ที่ 3S มีทางแยก คลังบริษัทพอข้าม 1S ไป 6S ไม่มีแหล่งใดพอไป 5A
- **จัดส่งสำเร็จ** ต้องครบ 3 เงื่อนไข ขนส่งส่งถึง แจ้งเลขติดตามแล้ว และตัดสต๊อกครบทุกรายการ

สไลด์ 17 Sequence diagram: MVC

ประมาณ 1 นาที

พูด

- ก่อนดูทีละ Use Case ขออธิบายวิธีอ่าน Sequence ในเล่ม ทุกรูปเขียนแบบ **MVC** ตามที่อาจารย์สอน
- Actor คือ Super Admin กดที่ **Screen** คือหน้าเว็บ `page.tsx` (ชั้น View ของ MVC) Screen ส่งคำสั่งให้ **Controller** คือ Server Action ใน `actions.ts` Controller เรียก **Model** คือ UseCase กับ Queries ที่เก็บ SQL แล้วเข้าฐานข้อมูลผ่าน `:DBConnector` ตัวเดียว
- ผลย้อนกลับเป็นเส้นประจนถึง Screen แล้ว Screen แสดงผลเองเป็น self message เช่น `showSuccess()` **จึงไม่มีลูกศรกลับไปหา Actor** และ Screen ไม่เรียกฐานข้อมูลตรง
- ชื่อ Screen และ Controller เป็นชื่อบทบาทที่จับคู่กับไฟล์จริง ในโค้ดไม่ได้สร้างคลาส Controller เพิ่ม

ชี้จอ ไล่กล่อง 5 กล่องด้านบนจากซ้ายไปขวา แล้วชี้บรรทัดสุดท้าย `Screen –>> Screen : showSuccess()`

วิธีพูด object ติด Actor ลงท้ายว่า Screen ถ้าถามให้พูดว่า “Screen ทำหน้าที่ View ของ MVC เกณฑ์ข้อ 24 ห้ามตั้งชื่อว่า view”

ตัวเลขที่ต้องเน้น **ไม่มีลูกศรกลับหา Actor**, **Screen → Controller → Model → DBConnector**

สไลด์ 18 1A Log in — ประมาณ 40 วินาที

พูด

- 1A มี 7 query `Q1A.1` อ่านบัญชีพร้อม `locked_until` ผิดครบ **5 ครั้ง** `Q1A.3` ล็อก **15 นาที** ระหว่างล็อกไม่ตรวจรหัสผ่านและบอกนาทีที่เหลือ ครบเวลา `Q1A.2` ปลดล็อกให้เอง
- สำเร็จ `Q1A.4` ล้างตัวนับ `Q1A.5` สร้าง session อายุ 8 ชั่วโมง ทางขวาคือส่วนแรกของ Sequence 1A (ในเล่มตัด 4 ส่วนให้ตัวอักษรใหญ่) Admin กดที่ LoginScreen ส่งให้ LoginController ไม่มีเส้นกลับหา Actor

สไลด์ 19 2A · 2S · 3A — ประมาณ 40 วินาที

พูด

- 2A Admin ยืนยันหรือแก้ SKU และจำนวน `Q2A.1` ล็อกแถว Order ก่อน
- 2S ระบบหา SKU จาก `channel_sku` ในตาราง skus ไม่มีตารางกฎแยก ไม่พบหรือพบหลายตัวส่งให้คนดู อ่านกฎไม่ได้ `Q2S.5` หยิบมาทำใหม่รอบถัดไป
- 3A ตั้งกฎ SKU บันทึกแล้ว `Q3A.8` ส่ง Order ที่ค้างกลับเข้าคิว และวิ่ง 2S 3S 6S ต่อใน transaction เดียวกัน

สไลด์ 20 3S Check stock — ประมาณ 55 วินาที

พูด

- 3S เป็นหัวใจของ Auto reorder มี 9 query `Q3S.1` ล็อกแถว stock ของ SKU `Q3S.2` อ่านยอด RSL `Q3S.3` หักยอดที่ Order อื่นจองไว้แต่ยังไม่ตัด `Q3S.4` บันทึกเส้นทางและ `fulfill_source`
- กฎ เลือก RSL ก่อน ถ้า RSL ว่างพอทั้งจำนวนไป 1S ถ้าไม่พอแต่คลังบริษัทพอไป 6S ทันที ถ้าไม่มีแหล่งใดพอทั้งจำนวนไปรอ Admin ตัดสินใจ 5A
- ไม่แบ่งส่งจากสองคลัง เพราะจะได้สองกล่องและค่าส่งสองครั้ง ถ้า RSL ล่มแสดงเฉพาะคลังบริษัทและบันทึก error

ชี้จอ กล่อง 3 กล่องทางขวาจากบนลงล่าง

ตัวเลขที่ต้องเน้น **RSL ก่อน**, **แต่ละแหล่งต้องพอทั้งจำนวน**

สไลด์ 21 3S Sequence diagram — ประมาณ 35 วินาที

พูด

ส่วนแรกของ Sequence 3S (ในเล่มมี 3 ส่วน) คือส่วนอัตโนมัติ เริ่มที่ ExecuteWorkflow หลัง 2S ตามโน้ตสีเหลือง เรียก CheckStockUseCase แล้ว StockQueries ลงฐานข้อมูลผ่าน DBConnector เห็นลำดับ ล็อก อ่าน RSL หักยอดจอง แล้วแยกทางในกรอบ alt หัวรูปมี Super Admin กับ StockCheckScreen เพราะส่วนที่ 2–3 คือหน้าตรวจสต๊อกที่ Admin เปิดเองผ่าน Screen และ Controller

สไลด์ 22 3S Collaboration diagram — ประมาณ 20 วินาที

พูด

Collaboration แปลงจาก Sequence object ตรงกันทุกตัว เลขลำดับแทนแกนเวลา ทางเลือกอยู่ใน

วงเล็บเหลี่ยม เราตรวจด้วยสคริปต์ว่าเลข query ใน Description, Sequence และ Collaboration ตรงกันครบทั้ง 17 Use Case

สไลด์ 23 1S · 6S · 9A Labels — ประมาณ 40 วินาที

พูด

- 1S จับคู่เลข RSL รับเฉพาะ Order ที่ 3S เลือก RSL
- 6S จัดรูปแบบใบปะหน้าที่ RSL ปกติ RSL-STD-A6 ของใหญ่ RSL-LARGE-A5 บริษัทส่งเอง INHOUSE-YAMATO-A6 ไม่มีเลข RSL
- 9A Admin กดพิมพ์ ทุกครั้งรวมพิมพ์ซ้ำบันทึกใน `label_print_history` การพิมพ์ยังให้คนกดเพราะเป็นงานกายภาพ

ชี้จอ ใบปะ RSL ด้านบน แล้วใบปะบริษัทส่งเองที่ช่อง RSL เป็น “—”

สไลด์ 24 4S Unit cost — ประมาณ 35 วินาที

พูด

- สูตรเดียวทุกครั้ง ราคาซื้อคูณอัตราแลกเปลี่ยน บวกค่าขนส่งระหว่างประเทศและภาษี หารจำนวนที่สั่ง แล้วบวกค่าธรรมเนียม Marketplace ค่าส่งในประเทศ และค่า RSL รวม 8 รายการ
- สูตรคำนวณในโค้ด SQL ใช้อ่านและบันทึก `Q4S.6` บันทึก `cost_history` ทุกครั้ง ค่าล่าสุดใน `costs` ใช้ต่อใน 5A

สไลด์ 25 5A Decide reorder — ประมาณ 40 วินาที

พูด

- 5A คือจุด human-in-the-loop มี 3 query `Q5A.1` ล็อก Order และดึงราคาขายกับต้นทุนล่าสุด ระบบแสดงแต่ไม่ตัดสินใจแทน
- เจ้าของกดคุ้มค่าพร้อมจำนวน หรือไม่คุ้มค่า `Q5A.2` บันทึก คุ้มค่าไป 6A ไม่คุ้มค่าไป 7A
- จำนวนไม่ใช่จำนวนเต็มบวกแจ้ง invalid-qty ไม่มีราคาขาย `Q5A.3` ส่งไปรอแก้ราคาที่ 3A

สไลด์ 26 5A Sequence diagram — ประมาณ 30 วินาที

พูด

Sequence ของ 5A สองส่วนวางคู่กัน ReorderScreen คือ Screen ติดกับ Actor ส่งคำสั่งให้ Reorder-Controller คือ Server Action ใน `actions.ts` ต่อไปที่ ReordersUseCase กับ ReorderQueries ลงฐานข้อมูลผ่าน DBConnector เส้นที่ลง DB มีเลข query ตรงกับ Description ผลย้อนกลับถึง Screen แล้ว Screen แสดงผลเอง ไม่มีเส้นกลับไปหา Actor

สไลด์ 27 5A Collaboration diagram — ประมาณ 20 วินาที

พูด

Collaboration ของ 5A ใช้ object ชุดเดียวกับ Sequence ไล่ตาม Screen, Controller, Model และ DB-Connector เลขลำดับแทนแกนเวลา และไม่มี message กลับไปหา Actor เช่นกัน

สไลด์ 28 6A Reorder and receive — ประมาณ 45 วินาที

พูด

- 6A มี 14 query สองช่วง ช่วงส่งใบสั่งซื้อ `Q6A.6` ขอเลขใบสั่งซื้อจาก sequence (`SELECT nextval('reorder_no_seq')`) ได้ PO-0092) `Q6A.7` สร้าง reorders `Q6A.8` ผูกทุก Order ที่รอ SKU นี้ผ่าน `orders.reorder_id` ใบเดียวครอบคลุมหลาย Order แล้วส่งให้ Supplier
- ช่วงรับของ `Q6A.10` ล็อกใบสั่งซื้อกับแถว stock `Q6A.11` เปลี่ยนเป็นได้รับสินค้าแล้วได้ครั้งเดียว `Q6A.12--13` เพิ่มสต๊อกคลังบริษัทพร้อมประวัติ `Q6A.14` ส่ง Order ที่ผูกไว้กลับไปตรวจสต๊อก แล้ว 3S กับ 6S ทำต่อทันที
- ทดสอบจริง รับ PO-0091 จำนวน 60 ชิ้น Order ที่ผูกไว้ 2 รายการกลับไปตรวจสต๊อก

สไลด์ 29 6A Sequence diagram — ประมาณ 30 วินาที

พูด

Sequence ของ 6A ยาวที่สุดในเล่ม ตัดเป็น 5 ส่วน ส่วนที่ 1–3 คือส่งใบสั่งซื้อผ่าน SupplierAdapter ที่เห็นนี้คือส่วนที่ 4 รับสินค้า Admin กดรับที่ ReorderScreen ส่งให้ ReorderController แล้ว UseCase วิ่งผ่าน ExecuteWorkflow ล็อกใบสั่งซื้อ `Q6A.10` เปลี่ยนสถานะ `Q6A.11` เพิ่มสต๊อก `Q6A.12` แล้ว 3S และ 6S ทำต่อใน transaction เดียวกัน

สไลด์ 30 6A Collaboration diagram — ประมาณ 15 วินาที

พูด

Collaboration ของ 6A ใช้ object ชุดเดียวกัน หน้าจอ ReorderScreen มีทั้งปุ่มสั่งซื้อและรับของ ส่งผ่าน Controller เลข Q6A.1 ถึง Q6A.14 ตรงกับ Description

สไลด์ 31 8A · 5S · 7S Ship and deduct — ประมาณ 45 วินาที

พูด

- 8A Admin ส่งมอบและบันทึกเลขติดตาม ไม่มีเลข Order รอที่ “รอส่งมอบ” เปิดหน้า `/shipping` ระบบดึงสถานะขนส่ง
- 5S เช็กประวัติก่อนส่งเพื่อไม่ส่งเลขซ้ำ ส่งผ่าน Rakuten adapter แล้ว 7S ต่อทันที
- 7S ตัดสต๊อกตามแหล่งที่ 3S เลือก และเข้าคิวอัปเดตยอดใน Rakuten ถ้า Rakuten ล่มตัดในระบบไว้ก่อน แล้วลองใหม่รอบถัดไป

สไลด์ 32 7S Sequence diagram — ประมาณ 25 วินาที

พูด

Sequence ของ 7S ส่วนแรกจาก 3 ส่วน เริ่มที่ ExecuteWorkflow ไม่มี Actor เพราะระบบทำเอง เริ่มจากรอบ retry `Q7S.1` หา Order ที่แจ้งเลขติดตามแล้วแต่ตัดสต๊อกไม่ครบ ครั้งละไม่เกิน 100 รายการ ข้ามแถวที่ถูกล็อก แล้ว `Q7S.2` ล็อก `Q7S.4` ลดยอด `Q7S.5` ตั้ง `stock_deducted` แล้วเข้าคิว sync Rakuten

สไลด์ 33 4A · 7A · 10A ประมาณ 35 วินาที

พูด

- 4A นำเข้า Order จาก Rakuten ข้าม Order ซ้ำ และบันทึกคำขอยกเลิกของลูกค้า
- 7A ต้องใส่เหตุผล คืนสต๊อกตามคลังที่ตัดไป ส่งมอบแล้วไป “รอดำเนินการพิเศษ” ถ้ายกเลิกใบสั่งซื้อด้วย Order อื่นที่ผูกไว้กลับไป 5A
- 10A ลบเฉพาะข้อมูลที่ปิดแล้ว เก็บ snapshot ลง `deletion_history` ก่อนลบเสมอ

ส่งต่อ “ต่อไปขอส่งให้นางสาวปิยากร ผลพานิช เล่าการออกแบบฐานข้อมูล สถาปัตยกรรม และการทดสอบครับ”

นางสาวปิยากร ผลพานิช

สไลด์ 34 ถึง 57

เวลาประมาณ 14 นาที 35 วินาที

สไลด์ 34 ER diagram

ประมาณ 40 วินาที

พูด

- ER แบบ Chen มี **17 entity** ตรงกับตารางจริงใน migrations 001 ถึง 008 **22 relationship** ทุกเส้น 1:1 หรือ 1:N ไม่มี M:N เส้นคู่คือ total participation
- reorders ครอบคลุม orders แบบ 1:N เก็บเป็น `orders.reorder_id` Order กับสินค้าแยกผ่าน order_items
- รายงาน รอบ แรก ออกแบบ 15 ตาราง พอ สร้าง เว็บ จริง ปรับ เป็น 17 เช่น เพิ่ม sessions, auto_error_logs และตารางประวัติ

สไลด์ 35 ER diagram with queries

ประมาณ 20 วินาที

พูด

ER พร้อม Query แสดงเฉพาะ PK และ FK ป้ายสีแดงคือเลข query ที่ใช้ entity นั้น ถ้าเป็น UPDATE แสดง attribute ที่ถูกแก้ด้วย รวม 113 query

สไลด์ 36 Tables and queries

ประมาณ 30 วินาที

พูด

- 2.17 มีโครงสร้างครบ 17 ตาราง attribute คำอธิบาย PK/FK NOT NULL ชนิดข้อมูล CHECK และตัวอย่าง 5 แถวต่อตาราง (admins มี 2 แถว เพราะมี Super Admin คนเดียว)
- 2.16 คือตาราง Query มีเลข query, Use Case และ SQL ทั้ง 113 ข้อ SQL คัดลอกจากค่าคงที่ในโค้ดตรงๆ

สไลด์ 37 Class diagram

ประมาณ 30 วินาที

พูด

- ได้จาก object ทุกตัวใน Sequence และ Collaboration แบ่ง 15 รูป 12 รูปตามกลุ่มงาน 3 รูปเป็นคลาสข้อมูล รูปนี้คือกลุ่ม 3S กับ 7S
- ชั้นตรงกับ MVC Screen (ชั้น View), Controller, UseCase, Queries, DBConnector และ Adapter ความสัมพันธ์มี association, dependency, aggregation และ realization

สไลด์ 38 State diagram: Order (1/3)

ประมาณ 35 วินาที

พูด

- อาจารย์ขอ State Diagram ของข้อมูลที่ถูก update เราทำครบ**ทุกคอลัมน์สถานะ**ในฐานข้อมูล รวม 5 คลาส

- เริ่มจาก `Order.order_status` 22 สถานะ แบ่งสามสไลด์ สไลด์นี้คือช่วงก่อนจัดส่ง ทุกเส้นเขียน [เงื่อนไข] event / Use Case พร้อมเลข query ที่ UPDATE สถานะ
- 3S ออกได้ 3 ทาง “สั่งซื้อแล้ว” ออกได้ 2 ทาง รับของ 6A หรือยกเลิกใบสั่งซื้อ 7A “รอดำเนินการด้วยตนเอง” มี `manual_reason` บอกสาเหตุ

สไลด์ 39 State diagram: Order (2/3) ประมาณ 30 วินาที

พูด

ช่วงจัดส่งในเล่มเป็นรูปเดียวแนวตั้ง บนสไลด์ตัดเป็นสองแถบให้อ่านได้ แถบบน Order ที่ใช้ RSL จับคู่เลข RSL ใน 1S แล้วจัดรูปแบบใบปะเป็น “รอพิมพ์ใบปะสินค้า” แถบล่าง 9A พิมพ์ใบปะ 8A ส่งมอบ มีเลขติดตามเป็น “อยู่ระหว่างจัดส่ง” ไม่มีเลขติดตามรอที่ “รอส่งมอบ”

ชี้จอ ชี้แถบบนก่อนแล้วไล่ลงแถบล่าง ซ้ายไปขวา

สไลด์ 40 State diagram: Order (3/3) ประมาณ 20 วินาที

พูด

5S แจ้งเลขแล้ว 7S ตัดสต๊อก “จัดส่งสำเร็จ” ต้องครบ 3 เงื่อนไข delivered แจ้งเลขแล้ว และตัดสต๊อกครบทุกรายการ ยกเลิกหลังส่งมอบ 7A ไป “รอดำเนินการพิเศษ”

ชี้จอ ชี้แถบสรุปสองกล่องใต้รูป

สไลด์ 41 State diagram: Reorder ประมาณ 30 วินาที

พูด

`Reorder.reorder_status` 6A สร้างใบสั่งซื้อ “รอส่งคำสั่งซื้อ” (`Q6A.7`) Supplier รับเป็น “สั่งซื้อแล้ว” (`Q6A.9`) ส่งไม่สำเร็จคงสถานะเดิม รับของ “ได้รับสินค้าแล้ว” (`Q6A.11`) ได้ครั้งเดียว ยกเลิกพร้อม Order ใน 7A (`Q7A.4`) และ 10A ลบเฉพาะใบที่ปิดแล้ว

สไลด์ 42 State diagram: Label · SKU ประมาณ 30 วินาที

พูด

`Label.label_status` 6S สร้างใบปะเป็น `ready` (`Q6S.4`) 9A พิมพ์สำเร็จเป็น `printed` (`Q9A.4`) ยกเลิกจับคู่ RSL ใน 1S ลบใบปะที่ยังไม่พิมพ์ `Sku.sku_status` ใน 3A สลับ `active/inactive` (`Q3A.9`) และลบได้เมื่อไม่มีข้อมูลอ้างถึง (`Q3A.11`)

สไลด์ 43 State diagram: delivery · sync ประมาณ 30 วินาที

พูด

`Order.delivery_status` ได้เลขติดตามใน 8A เป็น `in_transit` แล้วดึงสถานะขนส่งเป็น `delivered`, `failed` หรือ `returned` (`Q8A.4`) ส่วน `sync_status` ยอดสต๊อกเปลี่ยนจาก 3S 6A 7S เป็น `pending` แล้ว 7S ส่งยอดให้ Rakuten สำเร็จเป็น `synced` ไม่สำเร็จคง `pending` ไว้ลองใหม่

สไลด์ 44 Layers of the web app ประมาณ 40 วินาที

พูด

- เว็บที่สร้างจริงมี 5 ชั้นตาม MVC page ของ Next.js (ชั้น View ชื่อ Screen) → Server Action

(Controller) ใน `actions.ts` → UseCase → Queries ที่เก็บ SQL เป็นค่าคงที่ → `db.ts` ระบบภายนอกผ่าน Adapter

- ไม่มี REST API แยก ไม่มีคลาส Controller หรือ Repository (ชื่อ Screen/Controller ใน Sequence คือบทบาทของ page กับ Server Action) ไม่มี Scheduler ข้อมูลอยู่ใน PostgreSQL 17 ตาราง ไม่ใช่ localStorage

สไลด์ 45 executeWorkflow — ประมาณ 45 วินาที

พูด

- 2S 3S 6S 7S ทำงานผ่าน `executeWorkflow` เปิด transaction เดียว รันปุ่มที่ Admin กด แล้ว `runAutomation` ทำ Use Case อัตโนมัติของ Order นั้น และหยิบงานที่ค้างมาทำใหม่
- ถ้าขั้นอัตโนมัติพัง ROLLBACK ทั้งหมด แล้วบันทึก `auto_error_logs` ใน transaction แยก
- ไม่ใช้ Scheduler เพราะผลเห็นทันที ข้อมูลไม่ค้างครึ่งทาง และทุก action เป็นรอบ retry ถ้าใช้จริงเพิ่มตัวตั้งเวลาเรียก executeWorkflow ได้โดยไม่แก้ UseCase

สไลด์ 46 Locks and double deduction — ประมาณ 40 วินาที

พูด

- ทุกจุดที่แก้ข้อมูลสำคัญล็อกแถวก่อนด้วย `FOR UPDATE` ตามตารางซ้าย รอบ retry ใช้ `SKIP LOCKED` จะได้ไม่รอแถวที่ล็อกอยู่
- กันตัดสต๊อกซ้ำ 3 ชั้น `Q7S.2` ล็อกรายการกับ stock ตัดเฉพาะรายการที่ยังไม่ตัด และ `Q7S.5` UPDATE มีเงื่อนไข `stock_deducted = FALSE` สองรอบมาพร้อมกัน รอบที่สองเห็นว่าตัดแล้ว
- `Q7S.4` มีเงื่อนไขยอดต้องพอ สต๊อกจึงไม่ติดลบ

สไลด์ 47 External systems — ประมาณ 15 วินาที

พูด

ระบบภายนอก 6 interface Rakuten, RSL, Label Service, Printer, Carrier และ Supplier ตอนนี้เป็นตัวจำลองทั้งหมด ต่อของจริงเขียน implementation ตาม interface เดิม UseCase ไม่ต้องแก้

พูดเพิ่มถ้ามีเวลา

หน้า `/dev` สั่งให้ระบบภายนอกล่มได้ ใช้ทดสอบกรณีผิดพลาด

สไลด์ 48 Security — ประมาณ 25 วินาที

พูด

- รหัสผ่านเก็บแบบ scrypt มี salt สุ่ม เทียบแบบ timing safe
- session ใช้ token สุ่ม 32 byte ใน cookie ฐานข้อมูลเก็บแค่ SHA-256 ถ้าฐานข้อมูลรั่วก็เอา token ไปใช้ไม่ได้ cookie httpOnly อายุ 8 ชั่วโมง
- SQL ทุกข้อส่งค่าผ่าน parameter กัน SQL injection และล็อกบัญชี 15 นาทีเมื่อผิด 5 ครั้ง

สไลด์ 49 Platform and components — ประมาณ 10 วินาที

พูด

Next.js 16 กับ PostgreSQL ทางขวาคือ Component Diagram ชั้นเดียวกับสไลด์สถาปัตยกรรม

พูดเพิ่มถ้ามีเวลา

React 19, TypeScript, Tailwind และ shadcn ต่อฐานข้อมูลผ่าน node-postgres รัน SQL ในเล่มด้วย PGlite

สไลด์ 50 Site map and screens — ประมาณ 25 วินาที

พูด

Site map มี root คือ Login มี tree เดียวเพราะผู้ใช้ทุกคนเข้าทางเดียวกัน เมนูแบ่ง 5 กลุ่ม แต่ละหน้าจอคือ object Screen ใน Sequence ในเล่มมี 13 หน้าจอ Use Case ที่ระบบทำเองแสดงผลเป็นข้อความแจ้งเตือนในหน้าที่เปิดอยู่

สไลด์ 51 Output design — ประมาณ 20 วินาที

พูด

รายงาน 2 ฉบับ ใบปะสินค้าจาก 6S พิมพ์จาก 9A มี template 3 แบบ และรายงานต้นทุนต่อหน่วยจาก 4S ทั้งสองเปิดจากเว็บจริง

สไลด์ 52 SQL results — ประมาณ 20 วินาที

พูด

SQL **113 คำสั่ง** รันกับ PostgreSQL ผ่าน PGlite ผ่านทั้งหมด ทุกคำสั่งเริ่มจากข้อมูลตั้งต้นชุดเดียวกัน ตัวรันอยู่ในโฟลเดอร์เครื่องมือของเล่ม `04-chapter2-diagrams-database` ด้วย `npm run sql` SQL ทุกข้อคัดลอกจากค่าคงที่ใน `src/lib/queries` และจดชื่อค่าคงที่คู่กันไว้ เทียบทีละข้อแล้วตรงกัน

สไลด์ 53 Testing — ประมาณ 35 วินาที

พูด

- ทดสอบบนเว็บจริง **36 กรณี** ครบทุก Use Case ทั้งปกติและผิดพลาด ผ่านทั้งหมด ข้อความคาดหวังคัดจาก Description คำต่อคำ เช่น 3S บริษัทส่งเอง ORD-24194 และรับของ PO-0091
- ในโค้ดมี lint, type check, build, smoke test ของการ reset ส่วน SQL รันด้วย `npm run sql` ในโฟลเดอร์เครื่องมือของเล่ม `04-chapter2-diagrams-database`
- ยังไม่มี unit test หรือ e2e test อัตโนมัติ การทดสอบตามหน้าจอทำด้วยมือ

วิธีพูด พูดเรื่อง “ยังไม่มี” ตรงๆ ไม่ต้องขอโทษ

สไลด์ 54 Live demo + SELECT — ประมาณ 4 นาที

พูด

เปิด psql ไว้แล้ว “ขอสาธิตเว็บจริง ทุกขั้นจะ SELECT ก่อนกดและหลังกดให้เห็นว่าข้อมูลใน DB เปลี่ยนจริงค่ะ”

- 1 Login (1A) → SELECT `sessions` ได้แถวใหม่ เก็บแค่ `token_hash`
- 2 นำเข้า Order (4A) → SELECT นับตามสถานะ “รอตรวจสอบคำสั่งซื้อ” **7** → **8** (toast แจ้งเคสใน

seed เช่น ORD-24181 ลูกค้าขอยกเลิก SKU ไม่ลงทะเบียน เป็นเรื่องปกติ)

- 3 ยืนยัน ORD-24194 (2A) → SELECT order นี้ ระบบทำ 2S 3S 6S ต่อเอง สถานะ “รอพิมพ์ใบปะสินค้า” `fulfill_source = in_house` ใบปะ `ready`
- 4 พิมพ์ใบปะ (9A) → `label_status = printed` และ `label_print_history` 1 แถว
- 5 ส่งมอบ (8A) → “อยู่ระหว่างจัดส่ง” มีเลขติดตาม `delivery_status = in_transit` **สต๊อกยังไม่ตัด** เพราะ 7S ตัดหลัง 5S แจ้งเลขติดตาม
- 6 ตัดสินใจคุ้มค่า ORD-24184 (5A) → “รอสั่งซื้อจาก Supplier” (ตาราง `reorders` ยังไม่เปลี่ยน แถวใหม่เกิดตอนกดสั่งซื้อใน 6A)

ชี้จอ SQL อยู่บนสไลด์ 54 และในคู่มือหัวข้อเดโม พิมพ์ไว้ในไฟล์ แล้ว copy วางใน psql

วิธีพูด เวลาไม่พอ ทำแค่ข้อ 1–3 ถ้าพลาดกด reset ที่หน้า `/dev` ถ้าอาจารย์ให้เขียน SELECT เอง เริ่ม `SELECT * FROM <table> LIMIT 5;` แล้วเติม WHERE / JOIN ถ้าเว็บค้างเกิน 10 วินาที กลับสไลด์ 53 ชี้รูปผลทดสอบ

สไลด์ 55 Team and responsibilities — ประมาณ 20 วินาที

พูด

หน้าที่และคะแนนของแต่ละคนอยู่ในหัวข้อ 2.25 ของรายงาน ธนภัทรกับอชิระทำ Business Flow, ER, CRUD, Use Case Description และ Backend กับ Database ปิยากรประสานงานกับเจ้าของกิจการ เขียนรายงานหลัก ทำ Use Case Diagram, Sequence, Collaboration และ Frontend งานในเล่ม 16, 17 และ 20 ชิ้น คะแนนจากหัวหน้าทีม 5 เต็ม 5 ทุกคน

สไลด์ 56 Summary — ประมาณ 35 วินาที

พูด

- เดิมงานมือประมาณ 1 ถึง 2 ชั่วโมงต่อวัน ระบบใหม่ทำ 7 Use Case เอง ใบปะพร้อมทันที ต้นทุนจากสูตรเดียว แต่เจ้าของยังอนุมัติทุกการสั่งซื้อ
- งานในเล่ม 17 Use Case, 17 ตาราง, SQL 113 ข้อผ่าน, ทดสอบ 36 กรณีผ่าน
- ขั้นต่อไป ต่อ API จริงแทนตัวจำลอง เพิ่มตัวตั้งเวลาและ test อัตโนมัติ แล้วขยายช่องทางขาย

สไลด์ 57 Q&A — ประมาณ 10 วินาที

พูด

ขอบคุณค่ะ ยินดีตอบคำถาม

วิธีพูด คนที่พูดสไลด์นั้นเป็นคนตอบ เปิด `qna_prep/qna-Colorado.pdf` ไว้